

Principles of Concurrent Programming

Message Passing



SCUOLA
ALTI STUDI
LUCCA

Lorenzo Ceragioli



About Me

► Lorenzo Ceragioli (SySMA Research Unit)

Research Interests:

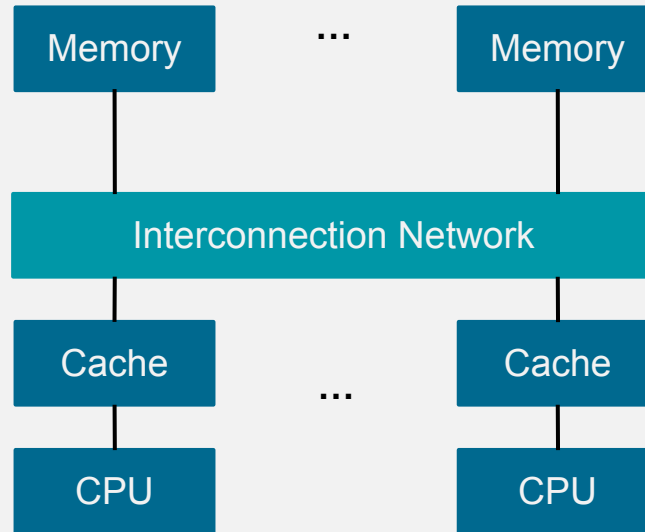
Verification of concurrent and interactive **quantum systems**
with Gadducci, Lomurno @ UniPI, Tedeschi @ IRIF

Formal methods for **computer security**
with Galletta, Lunati @ IMT Lucca

Contacts:

- Mail: lorenzo.ceragioli@imtlucca.it
- Page: lceragioli.github.io

Recall: Shared Variables Architecture



Why Message Passing

Distributed Memory Multiprocessor

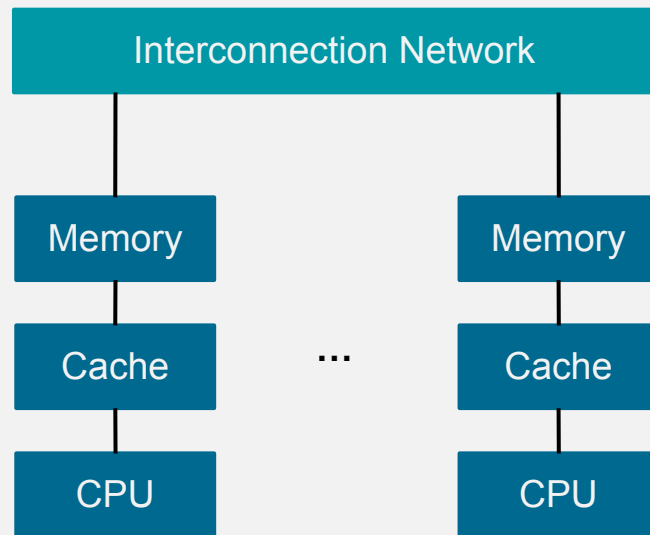
- variables are not shared
- interconnection network for message passing

Multicomputers

- processors are close

Network System

- processors are distant



Language Constructs and Computing Architecture

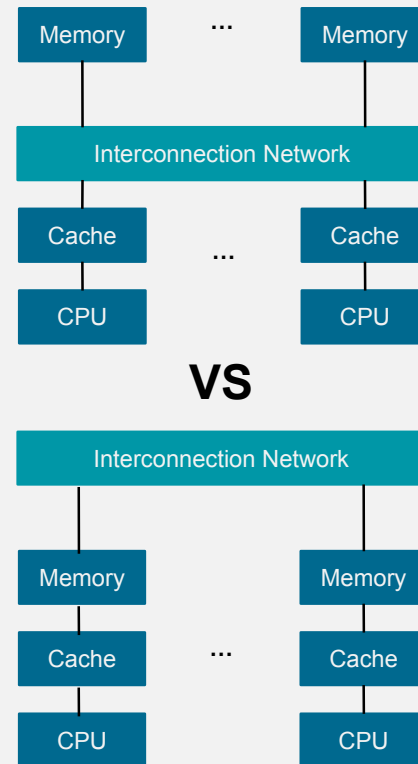
Language Constructs and Computing Architecture influence each other, but they are not the same!

Implementing **message passing** as a **programming abstraction** on a **shared variables architecture** is not difficult and can be useful for simplifying:

- writing code
- composing code
- verifying code

Implementing **shared memory** on a **network system** is more demanding but possible:

- allows automated distributed deployment
- Distributed Shared Memory (DSM)



Language Constructs and Semantics



Message Passing Basics

Two main ingredients

- Communication channels
- Send/Receive operations

Different possibilities:

- Receive/Send are blocking or nonblocking?
- Order of messages is preserved (also from different processes)?
- Buffers and Multiple Receivers
- Other operations like checking emptiness

(Value Passing) Calculus of Communicating Systems CCS

De facto standard process calculus

$P ::= \text{nil} \mid \tau.P \mid c!e.P \mid c?x.P \mid P + P \mid P \parallel P \mid P \setminus c \mid \text{if } e \text{ then } P \text{ else } P$

$e ::= \text{numerical expressions}$

$v ::= \text{numerical values}$

$c ::= \text{channels}$

Semantics is Given as a Labelled Transition system $(P, \text{Act}, \longrightarrow)$

- P is the set of processes, i.e. the states of the LTS
- Act are the actions, i.e. $\{\tau\} \cup \{!, ?\} \times \text{Chan} \times \text{Int}$
- $\longrightarrow$ is the transition relation, i.e. a subset of $P \times \text{Act} \times P$

Semantics of (Value Passing) CCS

We assume expressions are implicitly evaluated in advance ($e \mapsto v$)

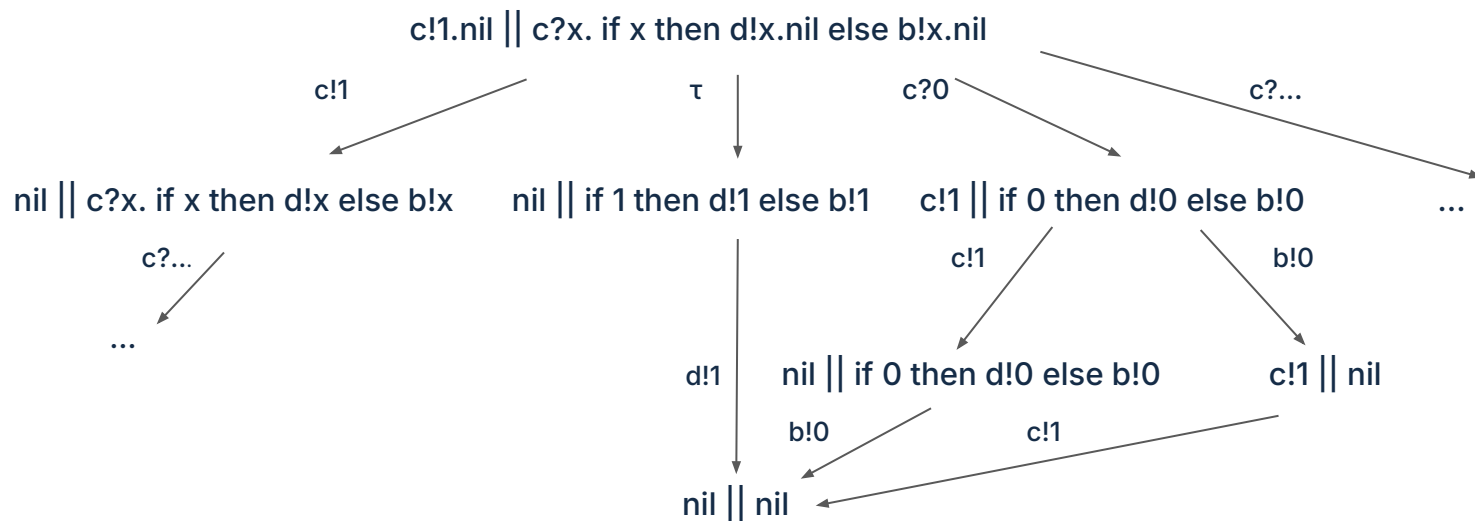
$$\frac{}{c!v.P \xrightarrow{c!v} P} \quad \frac{}{c?x.P \xrightarrow{c?v} P[v/x]}$$

$$\frac{P \xrightarrow{\mu} P'}{P + Q \xrightarrow{\mu} P'} \quad \frac{}{\tau.P \xrightarrow{\tau} P} \quad \frac{P \xrightarrow{\mu} P' \quad \mu \neq c!v, c?v}{P \setminus c \xrightarrow{\mu} P' \setminus c}$$

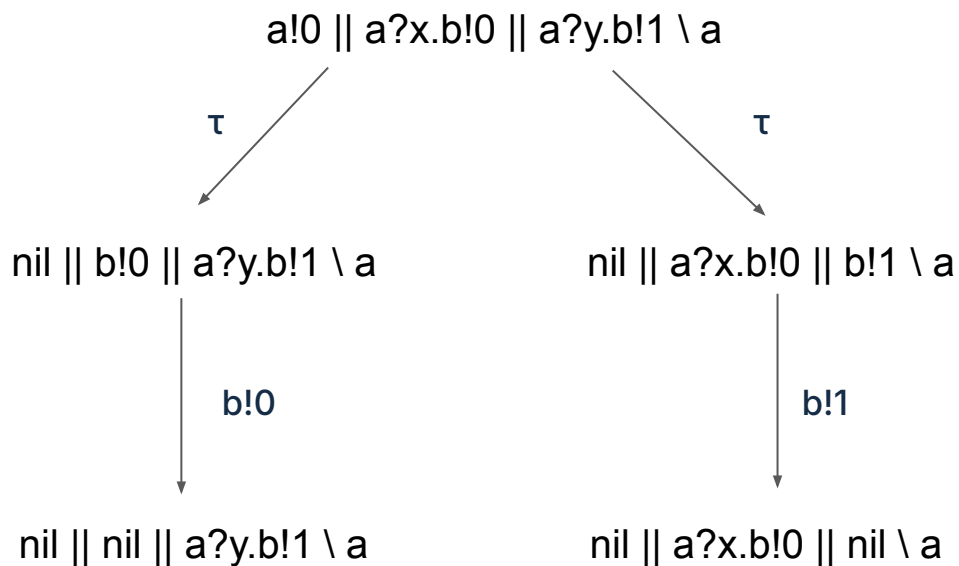
$$\frac{P \xrightarrow{\mu} P'}{P \parallel Q \xrightarrow{\mu} P' \parallel Q} \quad \frac{P \xrightarrow{c!v} P' \quad Q \xrightarrow{c?v} Q'}{P \parallel Q \xrightarrow{\tau} P' \parallel Q'} \quad \frac{n \neq 0 \quad P \xrightarrow{\mu} P'}{\text{if } n \text{ then } P \text{ else } Q \xrightarrow{\mu} P'}$$

Example

- τ -transitions encode the evolution of a closed system
- visible labels represent possible interactions with the environment



Example



Example

$a!0 \parallel a?x.c!x + b?x.d!x \setminus a \setminus b$

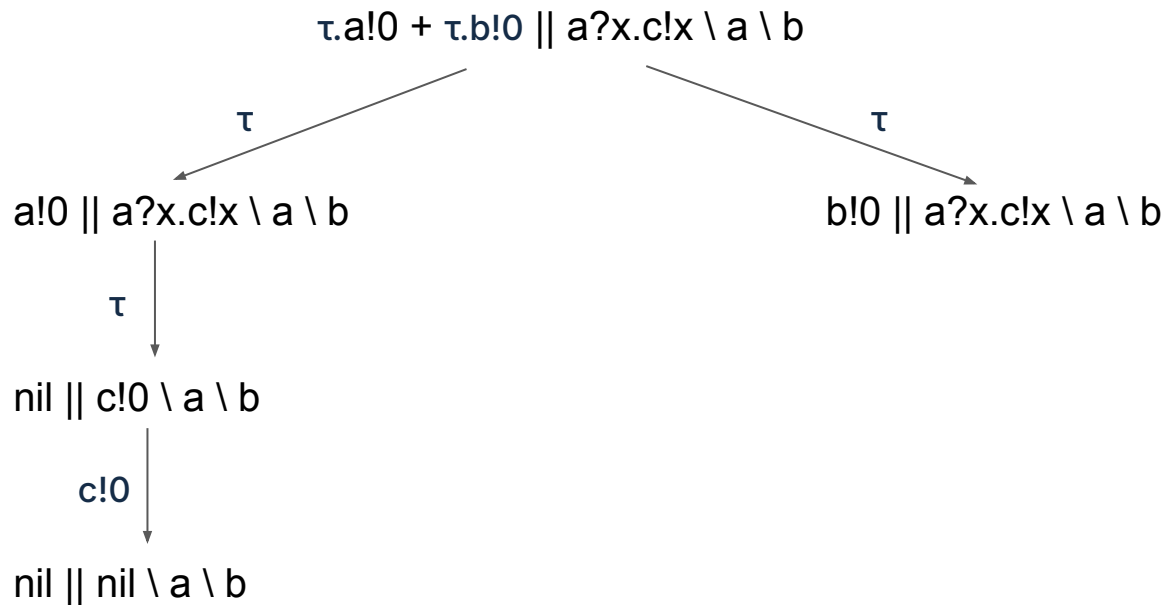
τ

$\text{nil} \parallel c!0 \setminus a \setminus b$

$c!0$

$\text{nil} \parallel \text{nil} \setminus a \setminus b$

Example



More Parallel Composition Operators

- $|||$ interleaving : $c!1.nil ||| c!2.nil = c!1.c!2.nil + c!2.c!1.nil$
- Handshaking: $(P || Q) \setminus c$
- selective input restriction: $(P || Q) \setminus c!$
- synchronous parallel: $c!1.P \otimes c!2.Q \xrightarrow{c!1 \otimes c!2} P \otimes Q$
- Asynchronous or buffered channels...

Standard CCS is Synchronous

Both send and receive are blocking: this is **Synchronous Message Passing**

- Not very common in distributed systems
- Can usually be encoded by asynchronous systems (using acks)
- Often conceptually simple, it is common in process calculi

Standard CCS is **Synchronous**

- When sending, the process is blocked until someone receive
- In the process **$c!1.d!2.nil$**
 - **$d!2$** is executed only if (and only after) sending 1 over c
- the process **$c!1.d!2.nil \parallel d?x.c?y.nil$**
 - has no tau-moves (deadlock)

Asynchronous Message Passing

Consider the following **assumptions**:

- Receive is blocking (as often)
- Send is nonblocking (common in practice)

This is an **Asynchronous System**!

Moreover we may **assume**:

- Sent Messages are eventually delivered
- The order of messages is preserved globally (channels are fifo queue)

Asynchrony Matters

In the synchronous case:

- P1 and P2 are in a deadlock
- You can solve this by forcing an order of execution
- Other solutions via non-determinism

In the asynchronous case:

- P1 and P2 communicate and shares the same variable values in the end

```
process P1 {  
    int v1 = 1, v2;  
    send ch12(v1);  
    receive ch21(v2);  
}
```

```
process P2 {  
    int v1, v2 = 2;  
    send ch21(v2);  
    receive ch12(v1);  
}
```

Asynchronous CCS

The idea is that

- sending operations cannot have a continuation
- the operational semantics is the same
- the intuitive meaning of $c!1$ differs
 - a sending operation in CCS
 - a sent message in ACCS

The example on the right is modelled as:

- $c!1.d!2.P$ in CCS
- $c!1 \parallel d!2 \parallel P$ in ACCS

```
process _ {  
    send c(1);  
    send d(2);  
    ...P...  
}
```

Note:

- $c!1 \parallel d!2 \parallel d?x.c?y.nil$ is not in deadlock!
- $c!1.d!2 \parallel d?x.c?y.nil$ is in deadlock in CCS, illegal in ACCS

Asynchronous CCS in Details

More in details, the syntax of ACCS is

$$P ::= Q \mid c!e \mid P \parallel P \mid P \setminus c \mid \text{if } e \text{ then } P \text{ else } P$$
$$Q ::= \text{nil} \mid \tau.P \mid c?x.P \mid Q + Q$$

- **$c!e$** has no continuations
- **$c!1 + d!2$** is illegal (external nondeterminism is for receptions)
- **$\tau.c!1 + \tau.d!2$** internal nondeterminism is possible
- processes can be rewritten as **$P1 \parallel P2 \dots \parallel PN \parallel c!e \parallel d!e \dots \parallel n!e$**

Asynchrony in LTS Models

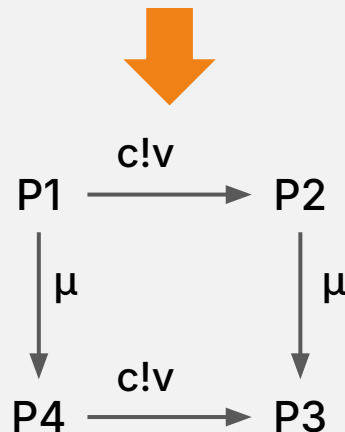
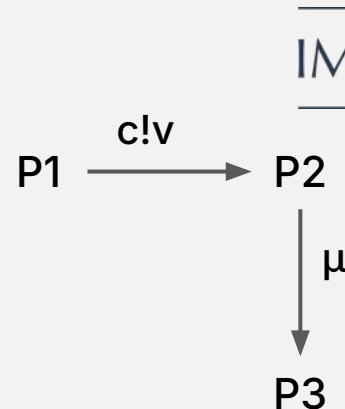
Asynchrony in the specification language:

- CCS $\longrightarrow$ ACCS (i.e. syntactic constraints)
- or new operational semantics

But the formal model is still an LTS

Semantic approach via **Selinger's axioms**:

- Axioms satisfied by asynchronous systems
- In theory, this allows deriving results that are not bounded by a specific process calculi or specification language



Communicating Sequential Processes (CSP)

CSP syntax: Named processes

```
process P1 { commands }  
process P2 { commands }  
...  
process Pn { commands }
```

Communication commands use the name of the target process:

P2!v means "send the value v to P2"

P3?x means "receive a value from P3 and store it into the variable x"

Other commands are rather standard

Communicating Sequential Processes (CSP)

An interesting feature is **Guarded Communication** (nondeterministic communication)

	if B1; C1 $\rightarrow$ S1;		do B1; C1 $\rightarrow$ S1;
act	[] B2; C2 $\rightarrow$ S2;		[] B2; C2 $\rightarrow$ S2;
just:	...	loop:	...
once	[] Bn; Cn $\rightarrow$ Sn;	forever	[] Bn; Cn $\rightarrow$ Sn;
	fi		od

Pass = { i | the boolean condition Bi holds }

Block = Pass $\cap$ { i | the communication Ci is **not** ready (wait) }

Ready = Pass $\cap$ { i | the communication Ci is ready (no wait) }

Semantics = $\left\{ \begin{array}{ll} \text{skip} & \text{if Pass is empty} \\ \text{exec from Ready} & \text{if Ready is not empty} \\ \text{wait and exec from Ready} & \text{Otherwise} \end{array} \right.$

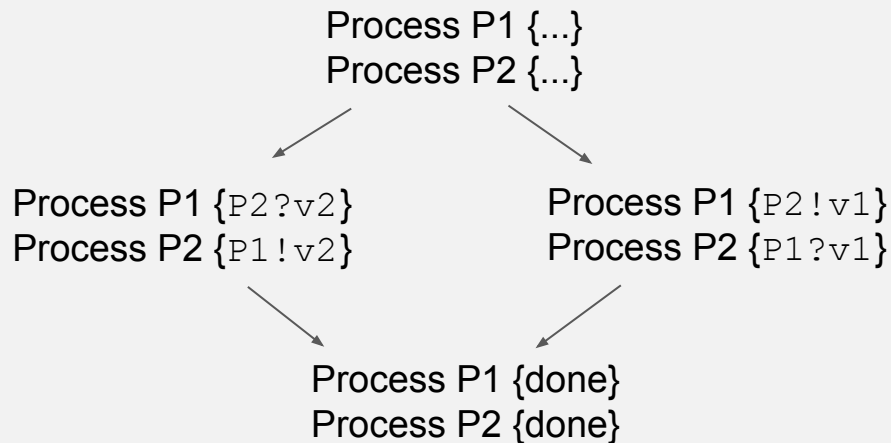
Example

```
process P1 {  
  int v1 = 1, v2;  
  if P2!v1 -> P2?v2;  
  [] P2?v2 -> P2!v1;  
  fi  
}
```

```
process P2 {  
  int v1, v2 = 2;  
  if P1!v2 -> P1?v1;  
  [] P1?v1 -> P1!v2;  
  fi  
}
```

Note:

- it avoids deadlock
- even in the synchronous case
- no ordering on communication



Different Types of Channels

In addition to Synchrony / Asynchrony:

- **Mailbox:** several senders and receivers
- **Port:** several senders, one receiver
- **Link:** one sender and one receiver

Even more: multicast, broadcast, singlecast

A Concrete Case: Go Language

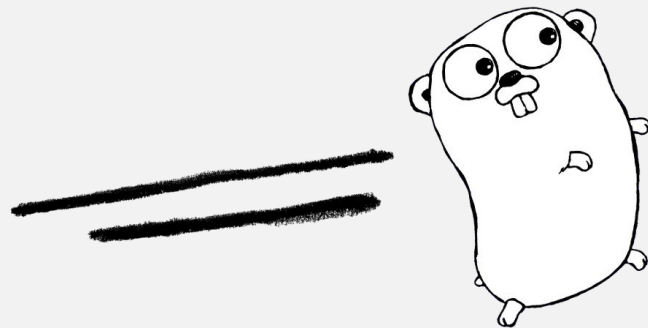


Go Language

A statically typed and compiled programming language designed at Google for high-performance networking and multiprocessing:

- syntactically similar to C
- garbage collection
- **built-in concurrency primitives**

CSP-style concurrency with light-weight processes (goroutines), channels, and the select statement



Goroutines

Lightweight threads, created with the `go` instruction:

- no guarantee on the order of execution
- any possible interleaving

The `defer` instruction defers the execution of a function until the surrounding function returns.

Termination:

- goroutines terminates when they have executed their code
- termination of the main function also kills all the goroutines

```
package main

import ("fmt"; "time")

func f(n int) {
    fmt.Println(n)
}

func g(){
    defer f(3)
    go f(1)
    go f(2)
}

func main() {
    defer f(4)
    go g()
    time.Sleep(10 * time.Second)
}
```

Shared Variables

Goroutines in a program share a single address space:

- mutable objects and pointers can be shared
- data race is possible
- standard library package featuring most of the classical concurrency control structures

But channels are preferred!

"Do not communicate by sharing memory; instead, share memory by communicating."

```
package main

import ("fmt"; "sync"; "time")

func inc(mu *sync.Mutex, v *int, d int) {
    mu.Lock()
    for j:=0; j<100; j++ {*v = *v+d}
    mu.Unlock()
}

func main() {
    var m sync.Mutex
    v := 0
    for i := 0; i < 1000; i++ {
        go inc(&m, &v, 1)
        go inc(&m, &v, -1)
    }
    time.Sleep(time.Second)
    fmt.Println(v)
}
```

Channels

Instruction for:

- **creation**

```
c1 := make(chan int) -
```

- **output**

```
c1 <- 5 -
```

- **input**

```
x := <- c1 -
```

- **closure**

```
close(c1) -
```

- **input with close check**

```
v, ok := <- ch -
```

```
func f1(c1 chan int, c2 chan int) {  
    v1 := 1  
    c1 <- v1  
    v2 := <- c2  
    fmt.Println(v1 + v2)  
}
```

```
func f2(c2 chan int, c1 chan int) {  
    v2 := 2  
    c2 <- v2  
    v1 := <- c1  
    fmt.Println(v1 + v2)  
}
```

```
func main() {  
    c1 := make(chan int)  
    c2 := make(chan int)  
    go f1(c1, c2)  
    go f2(c2, c1)  
    time.Sleep(10 * time.Second)  
}
```

Different Types of Channels

Synchrony by default: Sends and receives block until the other side is ready

```
c1 := make(chan int) -
```

Buffered channels: defined with a given buffer length, sends are blocking only when buffer is full, receive only when buffer is empty

```
c1 := make(chan int, 100) -
```

Technically Mailboxes: channel access is regulated by the visibility of its name

Select in Go

Like CSP Guarded Communication:

- nondeterministic communication
- but without boolean conditions

```
select {  
    case C1: S1  
    case C2: S2  
    ...  
    case Cn: Sn  
}
```

Effect:

- wait on multiple communication operations;
- it blocks until one of its cases can run, then it executes that case;
- It chooses one at random if multiple are ready

Example: from CSP to Google GO

```
process P1 {  
    int v1 = 1, v2;  
    if P2!v1 -> P2?v2;  
    [] P2?v2 -> P2!v1;  
    fi  
}
```

```
process P2 {  
    int v1, v2 = 2;  
    if P1!v2 -> P1?v1;  
    [] P1?v1 -> P1!v2;  
    fi  
}
```

```
func f1(c1 chan int, c2 chan int) {  
    v1 := 1  
    var v2 int  
    select {  
        case c1 <- v1: v2 = <- c2  
        case v2 = <- c2: c1 <- v1  
    }  
}
```

```
func f2(c2 chan int, c1 chan int) {  
    var v1 int  
    v2 := 2  
    select {  
        case c2 <- v2: v1 = <- c1  
        case v1 = <- c1: c2 <- v2  
    }  
}
```


Non-blocking Reception

Select with a default code: the default runs if no other case is ready

```
select {  
  case i := <- c:    // if you can receive over c now, proceed  
  default:          // if reception would block, do something else  
}
```

Usage: continuously waiting while doing something else:

```
for {  
  select {  
    case i := <- c:    // main computation  
    default:          // do something else while waiting  
  }  
}
```

Programming Patterns



Some Common Patterns

Producers and Consumers

Focus on data generation,
transformation and consuming

Clients and Servers

A process that serve a specific
job for multiple clients

Interacting Peers

Several processes jointly
performing a job

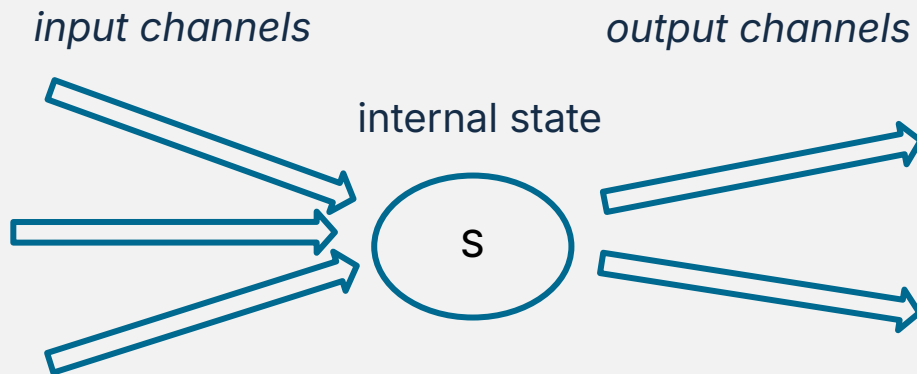
Producers and Consumers: Filters

Type system for channels defines legal compositions

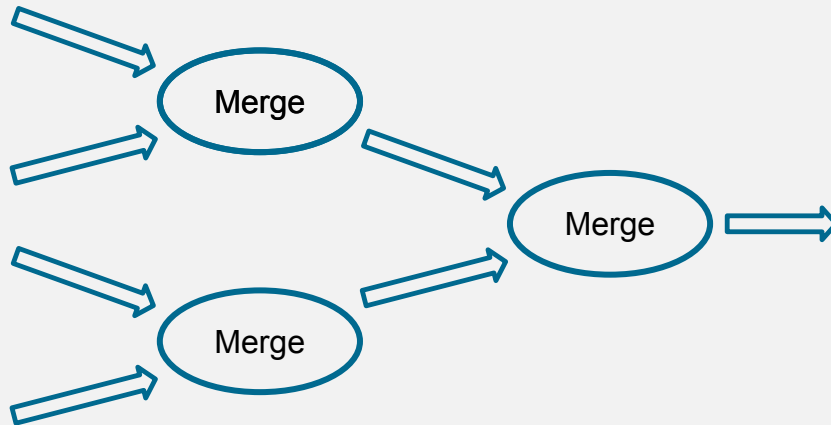
Asynchronous communication is efficient and natural

Defined by two functions over the input values and internal state:

- state update
- output value

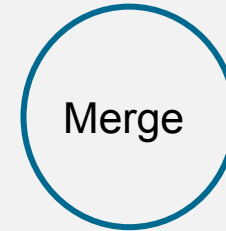


Filter Example: Sorting a Stream

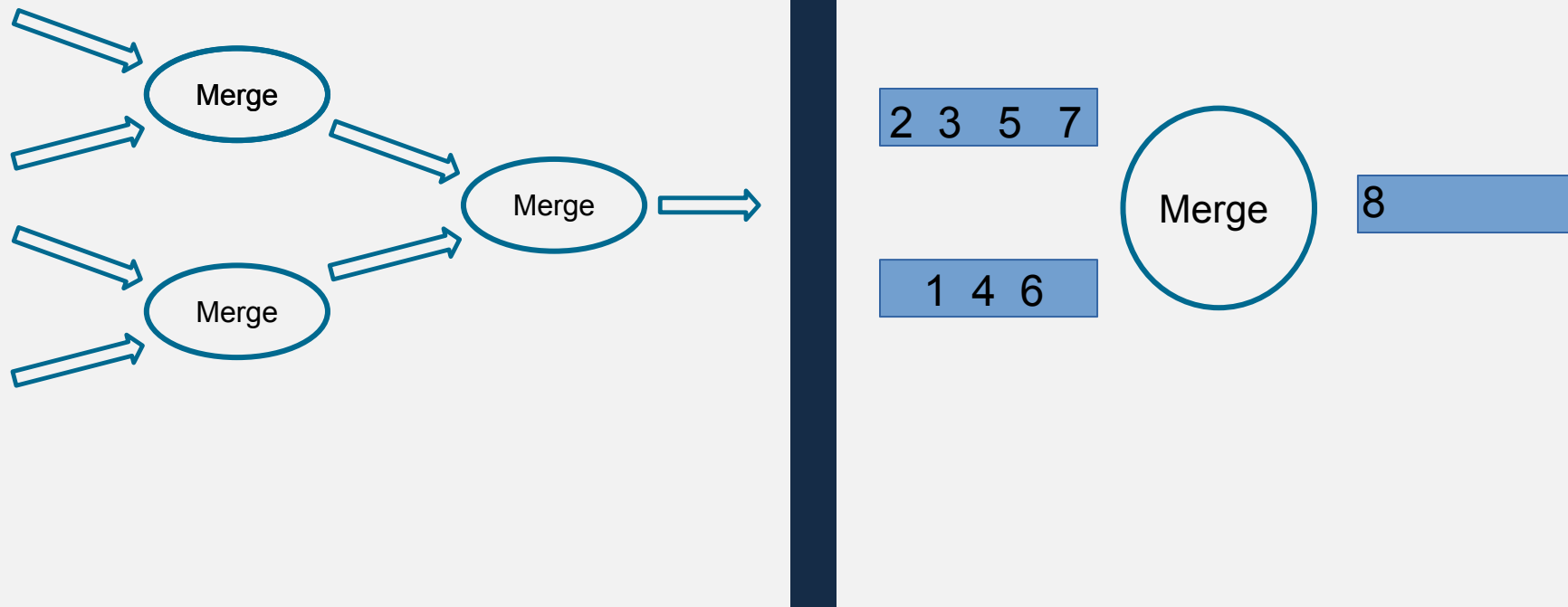


2 3 5 7

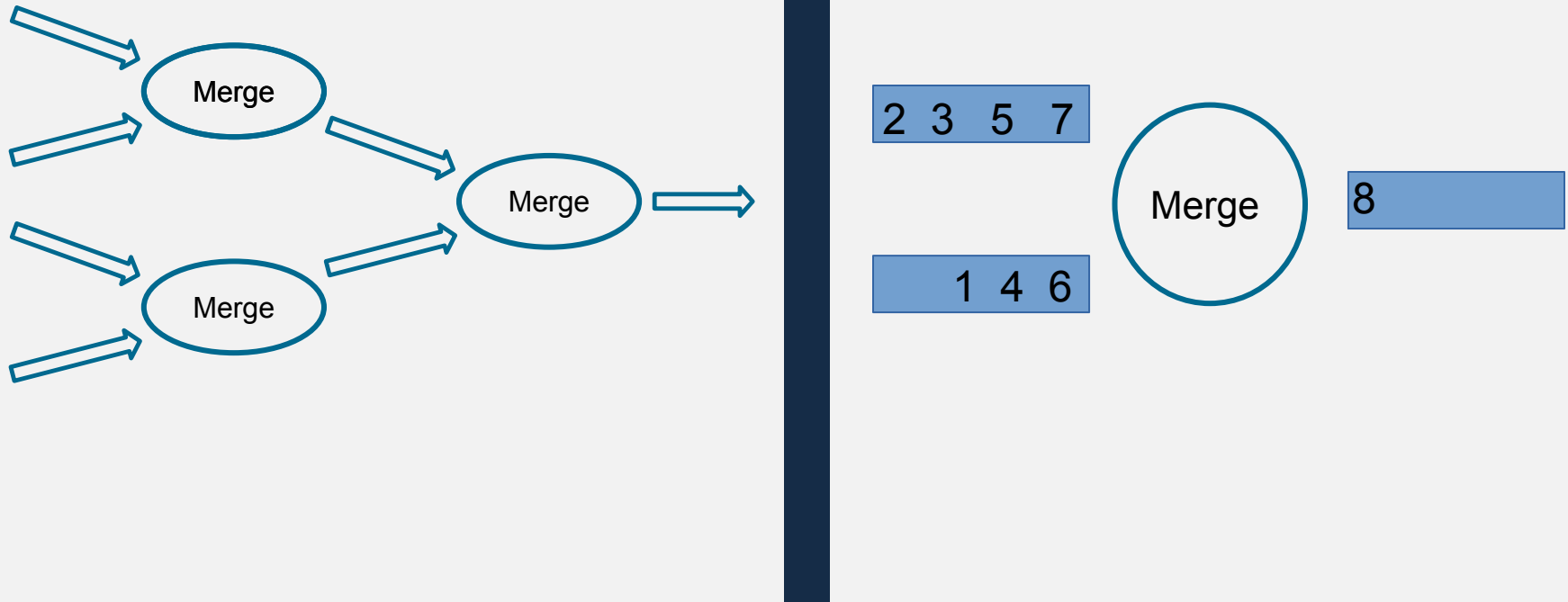
1 4 6 8



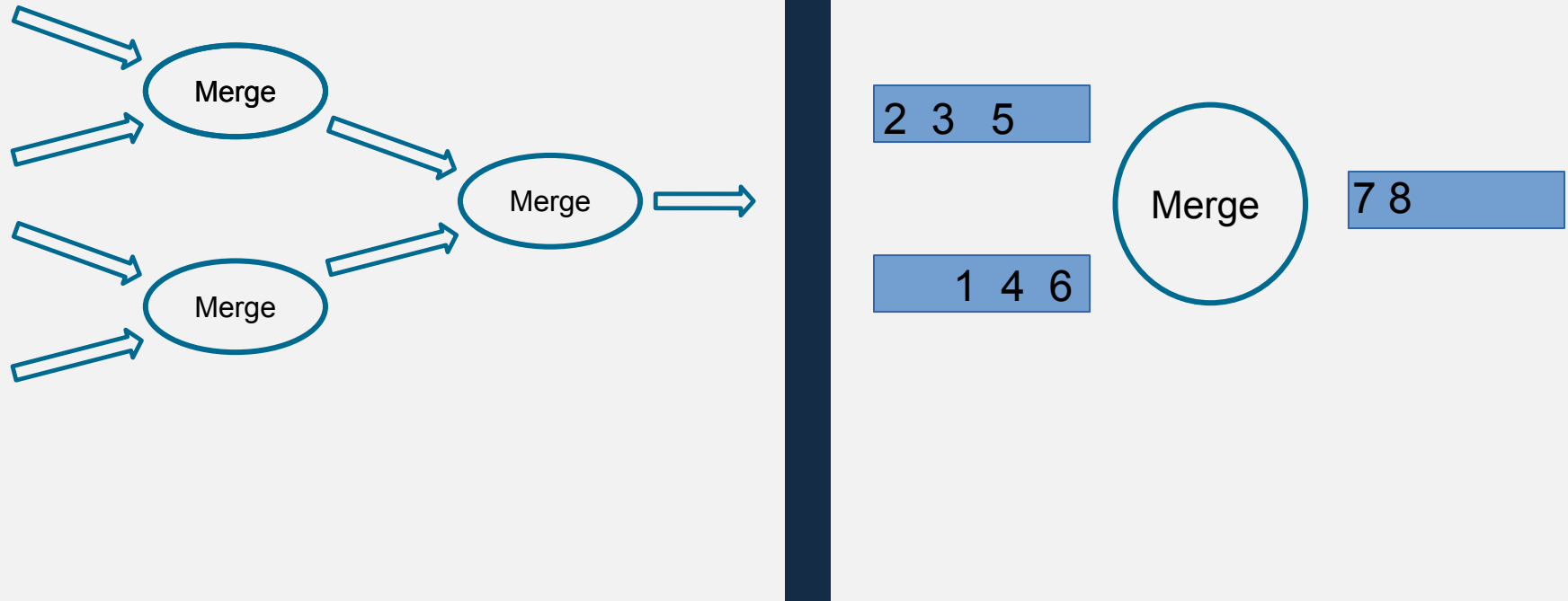
Filter Example: Sorting a Stream



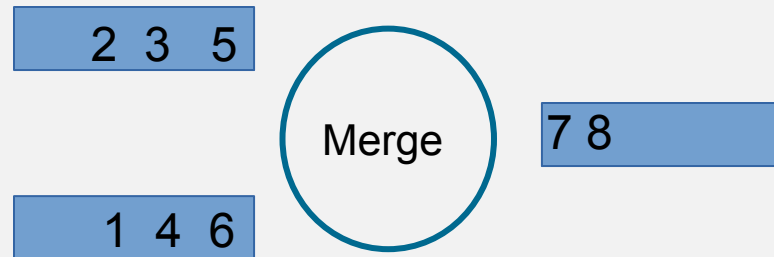
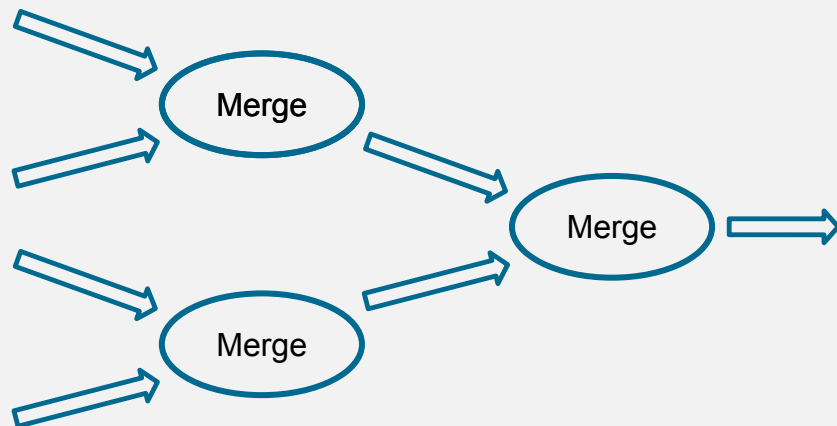
Filter Example: Sorting a Stream



Filter Example: Sorting a Stream



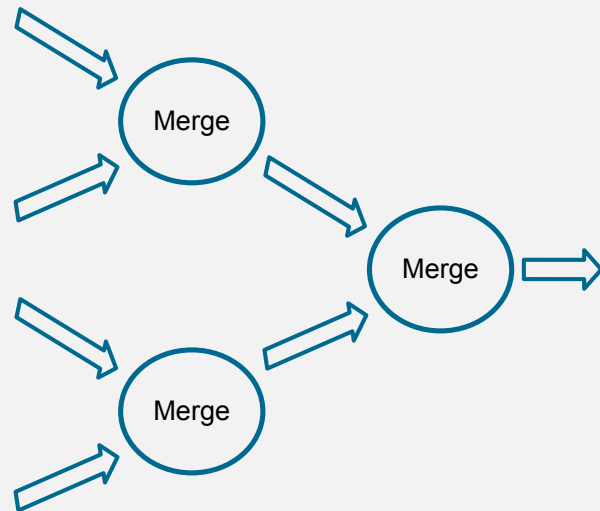
Filter Example: Sorting a Stream



Filter Example: Sorting a Stream

```
func sort(c1 chan int, c2 chan int, out chan int)
{
    v1, ok1 := <- c1
    v2, ok2 := <- c2

    for (ok1 && ok2) {
        if (v1 >= v2) { out <- v1
                        v1, ok1 = <- c1 }
        else { out <- v2
               v2, ok2 = <- c2 }}
    for (ok1) { out <- v1
                v1, ok1 = <- c1 }
    for (ok2) { out <- v2
                v2, ok2 = <- c2 }
    close(out)
}
```



Client and Server

Prominent in **WWW**

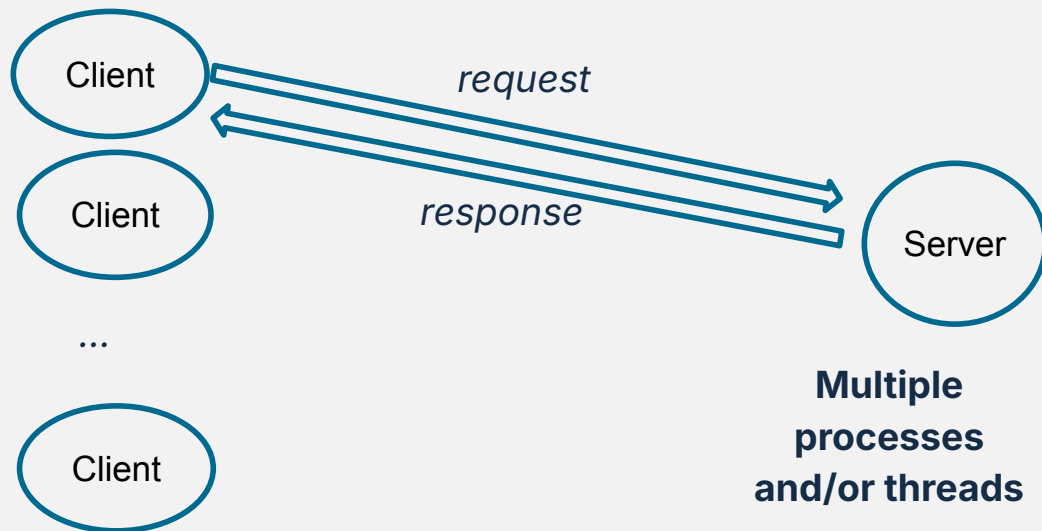
Concurrent Version of

- procedure (the server)
- procedure call (the client)

Guarantee conversational continuity

Inspired programming techniques:

- Remote Procedure Calls
- Rendezvous



From Monitors to Active Monitors

Monitors can be simulated using message passing

Passive Collection of Procedures	→ Active Process
Module Variables	→ Process Variables
Procedures Identifiers	→ Channels
Procedure Calls	→ Send Message and Receive Reply
Wait/Signal and Condition Variables	→ Explicit Save/Retrieve of Pending Requests

Remote Procedure Call

Client/Server interaction: concurrent version of procedures and procedure calls

RPC is a paradigm:

- client calls a procedure that is not local, but rather implemented in a server
- it hides the Client/Server interaction

Example: the client uses a function *f* in a module *M*

Client

```
main () {  
    x := M.f(4);  
}
```

Server

```
Module M {  
    f (x : int) { return x+3 }  
}
```

Remote Procedure Call – the actual behaviour

Client

```
main () {  
    x := M.f(4);  
}
```

Server

```
Module M {  
    f (x : int) { return x+3 }  
}
```

Remote Procedure Call – the actual behaviour

Client

```
main () {  
    x := M.f(4);  
}
```

```
Module M { // Stub  
    f (x : int) {  
        //code generated for communication  
        via message passing }  
}
```

Server

```
main () { // Skeleton  
    //code generated to wait  
    for and manage module M  
    related requests  
}
```

```
Module M {  
    f (x : int) { return x+3 }  
}
```

Remote Procedure Call – the actual behaviour

Client

```
main () {  
    x := M.f(4);  
}
```

```
Module M { // Stub  
    f (x : int) {  
        //code generated for communication  
        via message passing }  
}
```

Server

```
main () { // Skeleton  
    //code generated to wait  
    for and manage module M  
    related requests  
}
```

```
Module M {  
    f (x : int) { return x+3 }  
}
```


Remote Procedure Call – the actual behaviour

Client

```
main () {  
    x := M.f(4);  
}
```

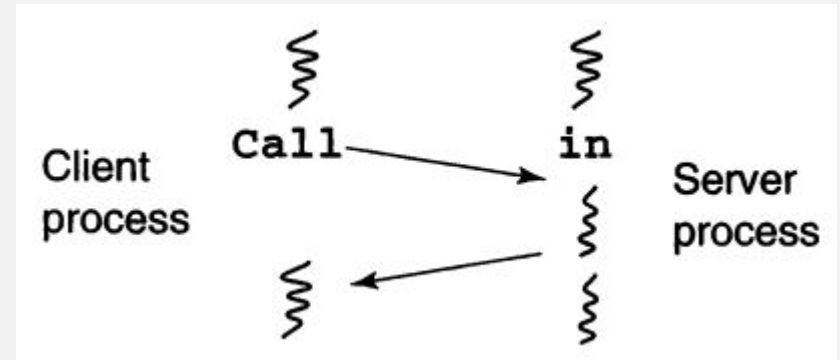
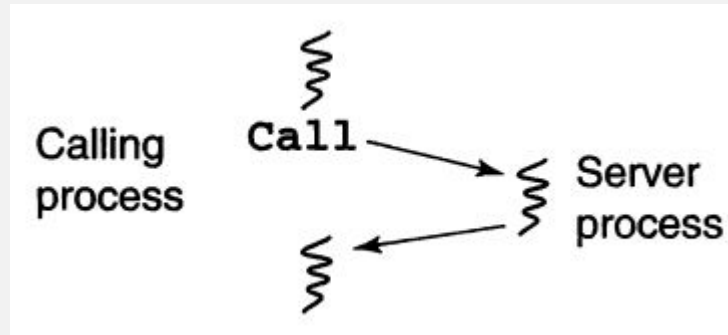
```
Module M { // Stub  
    f (x : int) {  
        //code generated for communication  
        via message passing }  
}
```

Server

```
main () { // Skeleton  
    //code generated to wait  
    for and manage module M  
    related requests  
}
```

```
Module M {  
    f (x : int) { return x+3 }  
}
```

Remote Procedure Call — Rendezvous



Several processes

- run the same code or are in the same role
- cooperate to accomplish a task
- often in distributed systems (e.g. P2P networks)

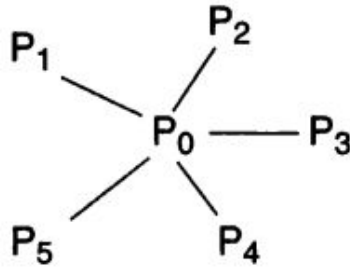
Several possible communication patterns: centralized, symmetric, ring, ...

Peer coordination is the most common relevant issue!

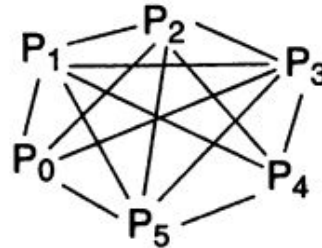
Interacting Peers: Example

Compute maximum of the values of each node

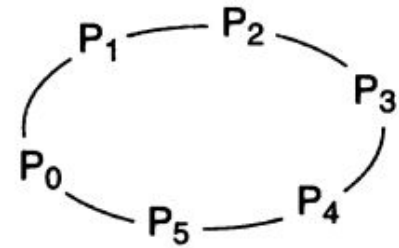
centralized



symmetric



ring



How to evaluate which one is better?

Modeling Concurrent Systems



LTS vs TS – CCS Reduction Semantics

Why not a reduction semantics?

Recall CCS

$P ::= \text{nil} \mid \tau.P \mid c!v.P \mid c?x.P \mid P + P \mid P \parallel P \mid P \setminus c \mid \text{if } e \text{ then } P \text{ else } P$

$e ::= \text{numerical expressions}$

$v ::= \text{numerical values}$

$c ::= \text{channels}$

Semantics is Given as a Labelled Transition system $(P, \text{Act}, \longrightarrow)$

P is the set of processes, i.e. the states of the LTS

Act are the actions, i.e. $\{\tau\} \cup (\{!, ?\} \times \text{Chan} \times \text{Int})$

$\longrightarrow$ is the transition relation, i.e. a subset of $P \times \text{Act} \times P$

Standard (Value Passing) CCS Semantics

$$\frac{}{c!v.P \xrightarrow{c!v} P} \quad \frac{}{c?x.P \xrightarrow{c?v} P[v/x]}$$

$$\frac{P \xrightarrow{\mu} P'}{P + Q \xrightarrow{\mu} P'} \quad \frac{}{\tau.P \xrightarrow{\tau} P} \quad \frac{P \xrightarrow{\mu} P' \quad \mu \neq c!v, c?v}{P \setminus c \xrightarrow{\mu} P' \setminus c}$$

$$\frac{P \xrightarrow{\mu} P'}{P \parallel Q \xrightarrow{\mu} P' \parallel Q} \quad \frac{P \xrightarrow{c!v} P' \quad Q \xrightarrow{c?v} Q'}{P \parallel Q \xrightarrow{\tau} P' \parallel Q'} \quad \frac{n = 0 \quad P \xrightarrow{\mu} P'}{\text{if } n \text{ then } P \text{ else } Q \xrightarrow{\mu} P'}$$

Alternative Approach: Reduction Semantics

$$\frac{P \rightarrow P'}{P + Q \rightarrow P'} \quad \overline{\tau.P \rightarrow P} \quad \frac{P \rightarrow P'}{P \setminus c \rightarrow P' \setminus c}$$

$$\overline{((c!v.P) + R) \parallel ((c?x.Q) + S) \rightarrow P \parallel Q[v/x]}$$

$$\frac{P \rightarrow P'}{P \parallel Q \rightarrow P' \parallel Q} \quad \frac{P \cong P' \quad P' \rightarrow Q \quad Q \cong Q'}{P \rightarrow Q'}$$

Reduction VS Labelled Semantics

Reduction	Labelled
Touchstone behaviour	Derived representation
Usually correct by construction	Usually more manageable
c!0.nil has the same semantics of nil	c!0.nil differs from nil
Often combined with contextual reasoning	Encode contexts as abstract environment
Not compositional	Compositional

Consistency usually via **full abstraction** of some **behavioural equivalence**

Labelled Semantics of P1 and P2 coincides
if and only if
Reduction Semantics of P1 and P2 coincides

Principles of Concurrent Programming

Message Passing

